Методические указания к лабораторной работе

Автор: доцент НИТПУ Друки А.А.

Шумоподавление на изображениях

Данная лабораторная работа посвящена шумоподавлению на изображении с помощью сверточного нейросетевого автоэнкодера.

Цель работы: получить навыки восстановления изображения путем удаления шумов с помощью сверточной нейронной сети класса автоэнкодер.

Задачи:

1. Ознакомиться с работой сверточных нейронных сетей класса автоэнкодеры.
2. Научиться решать задачу восстановления изображений путем шумоподавления.

Нейросетевые автоэнкодеры

Нейросетевой автоэнкодер - это тип искусственной нейронной сети, который используется для преобразования входных данных в скрытое представление, а затем обратно в выходные данные. Он состоит из двух частей: кодировщика (encoder) и декодировщика (decoder). Кодировщик принимает входные данные и преобразует их в скрытое представление, которое содержит информацию о ключевых признаках данных. Декодировщик затем берет это скрытое представление и восстанавливает исходные данные.

Автоэнкодеры могут использоваться в различных областях, включая компьютерное зрение, обработку естественного языка, рекомендательные системы и анализ данных. Они могут использоваться для сжатия данных, удаления шума из изображений или текста, генерации новых данных и многих других задач.

На рисунке 1 представлен пример работы автоэнкодера для задачи шумоподавления на изображениях.

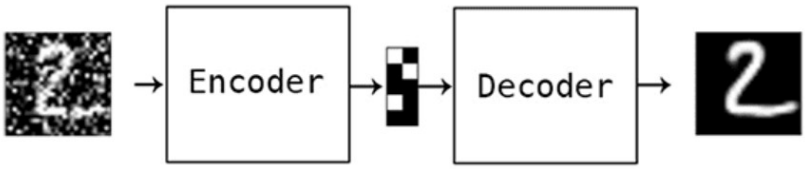


Рисунок 1. Пример работы автоэнкодера для шумоподавления на изображении

Ход работы

1. Загрузка данных

В качестве обучающей и тестовой выборки используется набор изображений рукописных цифр MNIST (Modified National Institute of Standards and Technology). MNIST является стандартом, предложенным Национальным институтом стандартов и технологий США с целью калибрации и сопоставления методов распознавания изображений с помощью машинного обучения в первую очередь на основе нейронных сетей. Данные состоят из заранее подготовленных примеров изображений, на основе которых проводится обучение и тестирование систем. Выборка разбита на 10 классов изображений рукописных цифр: 0, 1, 2 , 3, 4, 5 ,6, 7,

8, 9. Данная выборка содержит 60000 изображений размером 28x28 пикселей. На рисунке 2 представлены примеры изображений базы MNIST.

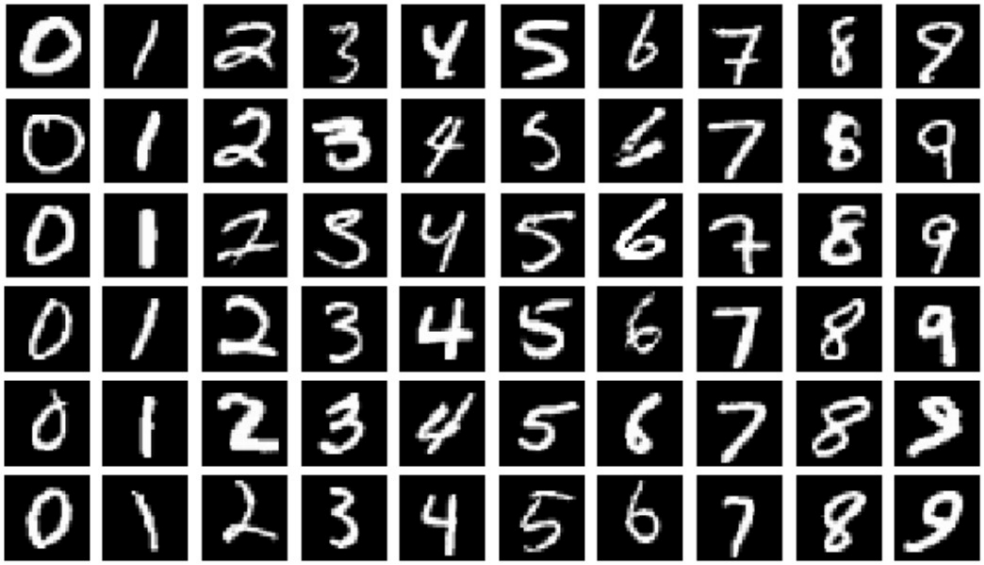


Рисунок 2. Пример изображений из базы MNIST

Для начала работы откройте Google Colab. Основы работы с данным сервисом вы получили в прошлых лабораторных работах.

В данной работе обязательно подключите графический процессор GPU к вашей среде разработки. Сделать это можно в настройках верхнего меню: «Среда выполнения» -> «Сменить среду выполнения» -> «T4 GPU». Это в несколько раз сократит время обучения вашей нейронной сети.

Выполним загрузку необходимых библиотек.

Numpy – это расширение языка Python, добавляющее поддержку больших многомерных массивов и матриц, вместе с большой библиотекой высокоуровневых математических функций для операций с этими массивами.

Matplotlib – библиотека для визуализации данных двумерной и трёхмерной графикой.

Keras – открытая нейросетевая библиотека для работы с нейронными сетями. Представляет собой надстройку над более низкоуровневыми фреймворками TensorFlow и Theano.

```
import numpy as np
import matplotlib.pyplot as plt

import keras
from keras import layers
from keras.datasets import mnist
from keras.models import Model
```

Загрузим базу изображений MNIST.

```
(x_train, _), (x_test, _) = mnist.load_data()
```

x_train – массив в который загружаются изображения из обучающей выборки.

x_test – массив в который загружаются изображения из тестовой выборки.

Так как для кодирования и декодирования нам нужны только оригинальные изображения и их варианты с наложением шума, то здесь мы не используем метки.

2. Предобработка данных

Для предобработки данных необходимо разделить данные в массивах x_train и x_test на число 255. Это выполняется для того, чтобы привести значения пикселей на изображениях в диапазон [0; 1]. Пиксели на изображениях имеют значения от 0 до 255, а выходные значения нейронной сети имеют значения в диапазоне от 0 до 1. Поэтому, для корректной работы сети мы приводим данные к одному формату.

```
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
```

Далее создадим изображения с шумом. В процессе обучения зашумленные изображения будут подаваться на вход нейронной сети, а на выходе ей будут предъявляться чистые изображения без шума в качестве целевых, что бы она стремилась к этому результату.

```
noise_factor = 0.9
x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0,
scale=1.0, size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0, scale=1.0,
size=x_test.shape)
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)
```

noise_factor – коэффициент степени зашумленности изображения. Чем он больше, тем более зашумленными получатся изображения. Можете определить это значение на свое усмотрение.

x_train_noisy – массив зашумленных изображений для обучения нейронной сети.

x_test_noisy – массив зашумленных изображений для тестирования нейронной сети.

3. Вывод загруженных изображений на экран

Для того чтобы проверить, правильность загрузки наших данных и посмотреть примеры изображений без шума и с наложенным шумом, выведем по 10 случайных изображений. Для вывода изображений на экран напишем функцию **display**, эта функция нам понадобится в дальнейшем, для оценки работы нейронной сети.

```
def display(array):
    n = 10
    plt.figure(figsize=(20, 2))
    for i in range(1, n + 1):
        ax = plt.subplot(1, n, i)
        plt.imshow(array[i].reshape(28, 28))
        plt.gray()
        ax.get_xaxis().set_visible(False)
        ax.get_yaxis().set_visible(False)
    plt.show()
```

Применим эту функцию к массивам x_test и x_test_noisy.

```
display(x_test)
```

```
display(x_test_noisy)
```

На рисунке 3 представлены выведенные на экран изображения рукописных цифр и соответствующие им зашумленные изображения.

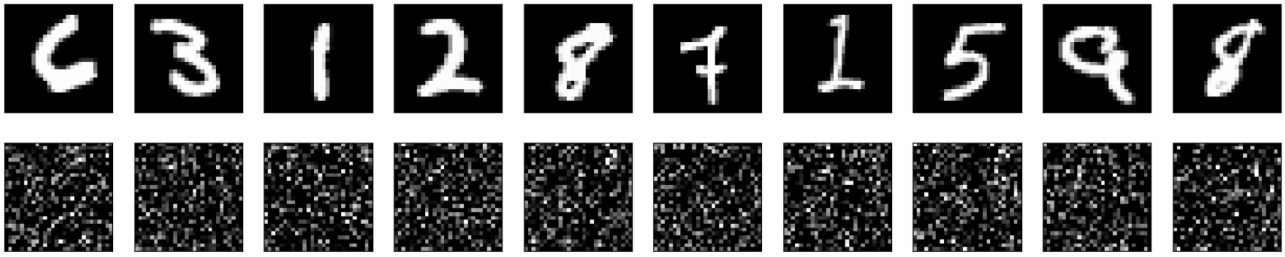


Рисунок 3. Вывод чистых и зашумленных изображений на экран

4. Реализация топологии автоэнкодера

На рисунке 4 представлена архитектура автоэнкодера в упрощенном виде Как видно по рисунку, сеть содержит сжимающий путь (encoder) и расширяющий путь (decoder). На вход сети поступает зашумленное изображение. Encoder извлекает из входного изображения небольшое количества самых важных ключевых признаков, всё лишнее отбрасывает. Далее по извлеченным ключевым признакам Decoder осуществляет восстановление изображения к исходному виду.

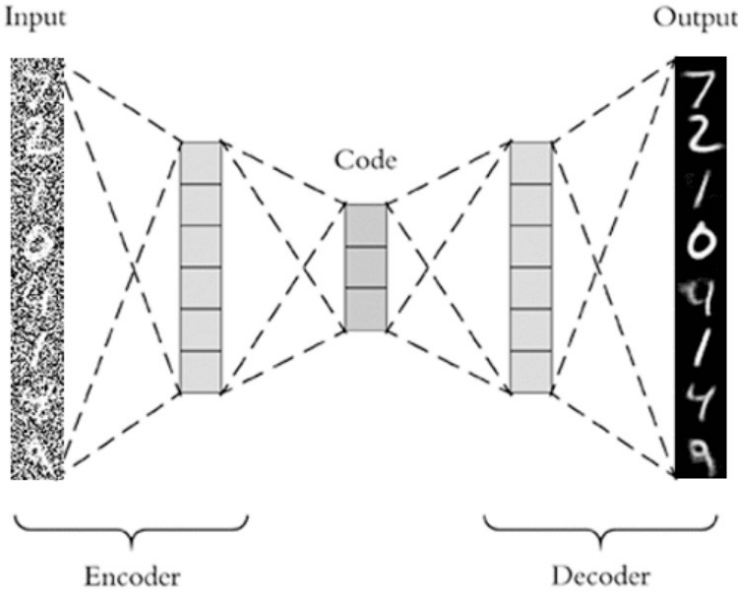


Рисунок 4. Архитектура классического автоэнкодера

В программном коде разделим архитектуру автоэнкодера на две части: encoder и decoder. По своей структуре и количеству параметров ни будут практически зеркальным отображением друг друга.

Encoder состоит из двух слоев Conv2D и двух MaxPooling2D. Эти слои чередуются друг с другом. В слоях Conv2D используется функция активации ReLU и padding. За счет использования слоев MaxPooling2D происходит постепенное уменьшение размера входного изображения.

Decoder аналогично состоит из двух слоев Conv2D, только вместо MaxPooling2D здесь используется UpSampling2D. Этот слой выполняет по сути обратную функцию к MaxPooling2D и увеличивает изображение в 2 раза.

Самый последний слой содержит функцию активации sigmoid.

Для реализации автоэнкодера будем использовать функциональный подход.

```
.
.
Допишите код
.
.
```

Conv2D – создание сверточного слоя со следующими параметрами: количество плоскостей (фильтров), размер ядра свертки, **activation** – функция активации, **padding** – параметр "same" означает не изменять размер изображения после его обработки сверточным слоем.

MaxPooling2D – создание слоя MaxPooling. Окном 2x2 сканируется предыдущий слой, выбирает в каждой такой области максимальный элемент и записывает его в текущий слой. Уменьшает размер входного слоя в 2 раза.

UpSampling2D – операция обратная MaxPooling. Слой увеличивается в 2 раза относительно предыдущего слоя.

ReLU – функция активации, выполняющая простое нелинейное преобразование поданных на вход данных (x). Возвращает x, если x > 0 и 0 в противном случае. Отличается высокой скоростью вычисления.

Softmax – функция активации, выполняющая нелинейное преобразование на основе математической функции.

5. Обучение нейронной сети

Для обучения нейронной сети необходимо вызвать метод .fit(), используя блок кода ниже:

```
.
.
Допишите код
.
.
```

x_train_noisy – входные данные (зашумленные изображения).

x_train – целевые данные (изображения без шума).

epochs – количество повторений циклов обучения для всей выборки данных.

shuffle – периодическое перемешивание данных в процессе обучения, что бы они поступали нейронную сеть в случайном порядке.

batch_size – размер пакета данных который будет пропускаться через нейронную сеть, и только после этого будут изменяться веса модели.

6. Тестирование нейронной сети

Для тестирования нейронной сети необходимо запустить следующий блок кода:

```
.
.
Допишите код
.
.
```

7. Результат работы нейронной сети

Чтобы сегментировать изображения, запустите следующий код:

```
.
.
Допишите код
.
.
```

Чтобы вывести результат работы нейронной сети применим ранее написанную функцию **display** к массиву **x_test_noisy** (зашумленные изображения, которые подаются на вход нейронной сети) и **predictions** (результат восстановления изображений и очистки их от шумов).

```
display(x_test_noisy)
display(predictions)
```

Результат очистки изображений от шумов представлен на рисунке 5.

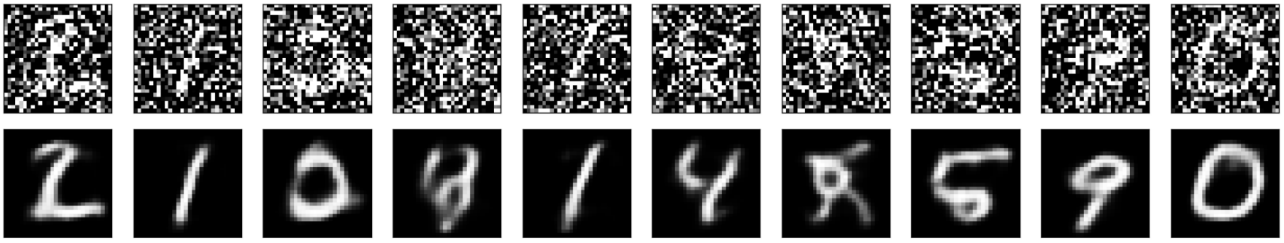


Рисунок 5. Результат очистки изображений от шумов

Как можно заметить по результату работы автоэнкодера, некоторые изображения не в полной мере восстановились в свое исходное состояние. Видны некоторые искажения. В некоторых случаях даже сложно определить цифру на восстановленном изображении. Это связано с большим коэффициентом степени зашумленности изображения **noise_factor**. Если его значение уменьшить, то результат улучшится.

Вывод

В результате выполнения данной лабораторной работы Вы познакомились с работой сверточных нейронных сетей класса автоэнкодеры и получили навыки очистки изображений от шумов.

Критерии оценки

Максимальное число баллов, которое можно получить по лабораторной работе – 8 баллов.

Минимальное число баллов, при котором задание засчитывается – 5 баллов.

Выполнение задания

Выполнено меньше половины задания – 0 баллов.

Большей частью выполнено – 2 балл.

Выполнено полностью – 4 балла.

Качество отчета

Сильные огрехи в оформлении или удовлетворяет менее половины требований к отчету – 0 баллов.

Хорошее оформление. Удовлетворяет большей части требований – 2 балл.

Соответствует всем требованиям – 4 балла.